Fashion MNIST CNN design and results

Exercise 1

Data and model structure

Each input is a 28 x 28 grayscale image. I use 54,000 images for training, 6,000 for validation and 10,000 for testing. Validation helps me choose the model weights. The test set measures performance on images not used to update the weights.

I divide pixel values by 255 to put them between 0 and 1. I one-hot encode each label into ten values and add one grayscale channel before the first convolution. Seed 42 makes the data split repeatable.

Stage	Main settings	Output per image
Convolution 1	32 filters, 3 x 3, stride 1, GELU	28 x 28 x 32
Max pooling 1	2 x 2, stride 2	14 x 14 x 32
Convolution 2	64 filters, 3 x 3, stride 1, GELU	14 x 14 x 64
Max pooling 2	2 x 2, stride 2	7 x 7 x 64
Convolution 3	64 filters, 3 x 3, stride 1, GELU	7 x 7 x 64
Classifier	Flatten, Dense 128, GELU, dropout 0.3	128 values
Output	Dense 10, softmax	10 probabilities

All three convolutions use one-pixel replicate padding.

Why use these convolutions

Three layers and 3 x 3 kernels. I want to learn local patterns first and then combine them in deeper layers. A 3 x 3 kernel examines a position and its immediate neighbors. Deeper layers combine information from a larger area of the original image.

Stride 1. The filter moves one position at a time. I avoid reducing the image size during convolution and leave spatial reduction to pooling. This does not mean that every detail is preserved.

Why chose the remaining values

32 then 64 then 64 filters. The first layer works on larger 28 x 28 maps. Using 32 filters instead of 64 there reduces computation and memory, with the aim of saving time. After pooling, the maps are smaller: 14 x 14 and 7 x 7. I use 64 filters in those layers to learn more feature combinations. Actual speed also depends on the number of input channels.

Replicate padding. I copy the nearest edge value into a one-pixel border. This lets the filter examine neighborhoods centered on the image boundary without inserting a zero border. It keeps the spatial size after convolution. The copied values are an assumption about the boundary, not new image information.

GELU activation. I chose GELU because it is smooth and can pass gradients for negative inputs. ReLU has a zero gradient for negative inputs. GELU may help learning in this region, but it does not guarantee better results. ReLU also supports backpropagation, and GELU gradients can still become very small.

2 x 2 max pooling. I keep the largest response in each small neighborhood. Pooling with stride 2 reduces 28 to 14 and then 14 to 7. I stop pooling at 7 x 7 to keep more spatial detail before classification. Max pooling keeps the strongest response, which is moslty the most useful information.

Dense 128 and dropout 0.3. The dense layer combines the 3,136 flattened features. I retained 128 hidden units as a starting size and used GELU here too. Dropout switches off 30% of these activations during training in order to transfer a part of the information of the dropout switches to the rest. This help us when some value missing from a test to not loss it information

Ten softmax outputs. There are ten clothing classes, so the final layer produces ten probabilities. I use categorical cross-entropy because the labels are one-hot encoded. Lower loss means that the model assigns more probability to the correct answers.

Training choices

Batch size 128. Assignment 1 used Iris, which has only 150 examples in total. Here I train on 54,000 images. Increasing the batch size from 32 to 128 reduces weight updates from 1,688 to 422 per epoch. Each update averages more examples, aiming for less noisy gradients and less update overhead. Smoother curves and better accuracy are not guaranteed.

Forty epochs. I kept the epoch count from Assignment 1. This lets us see both the early improvement and what happens when training continues for longer. The full run uses all 40 epochs.

Adam with learning rate 0.001. I retained these as starting training settings. Adam adjusts updates using information from earlier gradients. I did not compare different optimizers or learning rates as the exercise requested.

The model with the lowest validation loss is kept in memory. Test values are recorded after every epoch for the graphs, but they do not update the weights or choose the best model. Seed 42 is also used for training repeatability.

Learning curves and overfitting

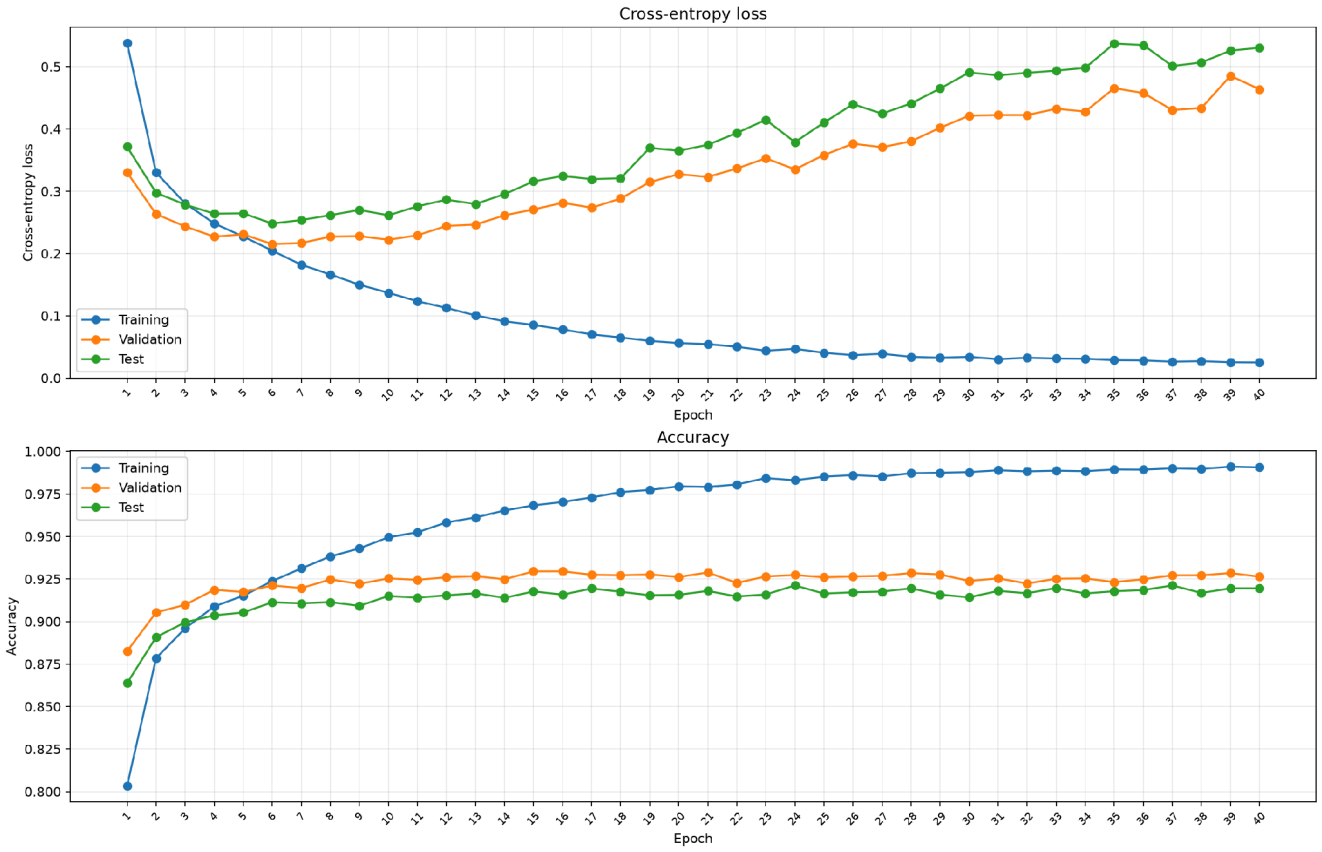


Figure 1. Loss and accuracy for training, validation and test data across all 40 epochs.

During roughly the first six epochs, all three losses fall. This means that the model improves on both the training data and unseen images. Around epoch 6, validation loss is near its lowest point, at about 0.21, while test loss is about 0.25. These values are approximate readings from the graph.

After this point, training loss continues to fall, but validation and test loss generally rise, with some short fluctuations. By epoch 40, training loss is about 0.03, validation loss about 0.46 and test loss about 0.53. The model fits the training examples more closely while its probability predictions on unseen images get worse. This is evidence of overfitting from around epoch 6.

Why accuracy does not show this as clearly

Accuracy only checks whether the class with the highest probability is correct. Cross-entropy also checks how much probability the model gives the correct class. For one image, the loss is minus the natural logarithm of that probability.

For example, suppose a shirt is wrongly predicted as a coat. If the probability of the correct shirt class falls from 0.20 to 0.02, both predictions are still wrong, so accuracy does not change. However, the loss for the 0.2 is much smaller than the loss for 0.02 probability. A few very confident mistakes can raise the average loss while accuracy stays similar.

Validation accuracy stays around 92 to 93 percent and test accuracy around 91 to 92 percent for much of the run. They do not show the same clear decline as the loss curves. However, training accuracy rises towards 99 percent, so the growing accuracy gap is also a warning sign. Loss gives the clearer signal here.

Test predictions by clothing class

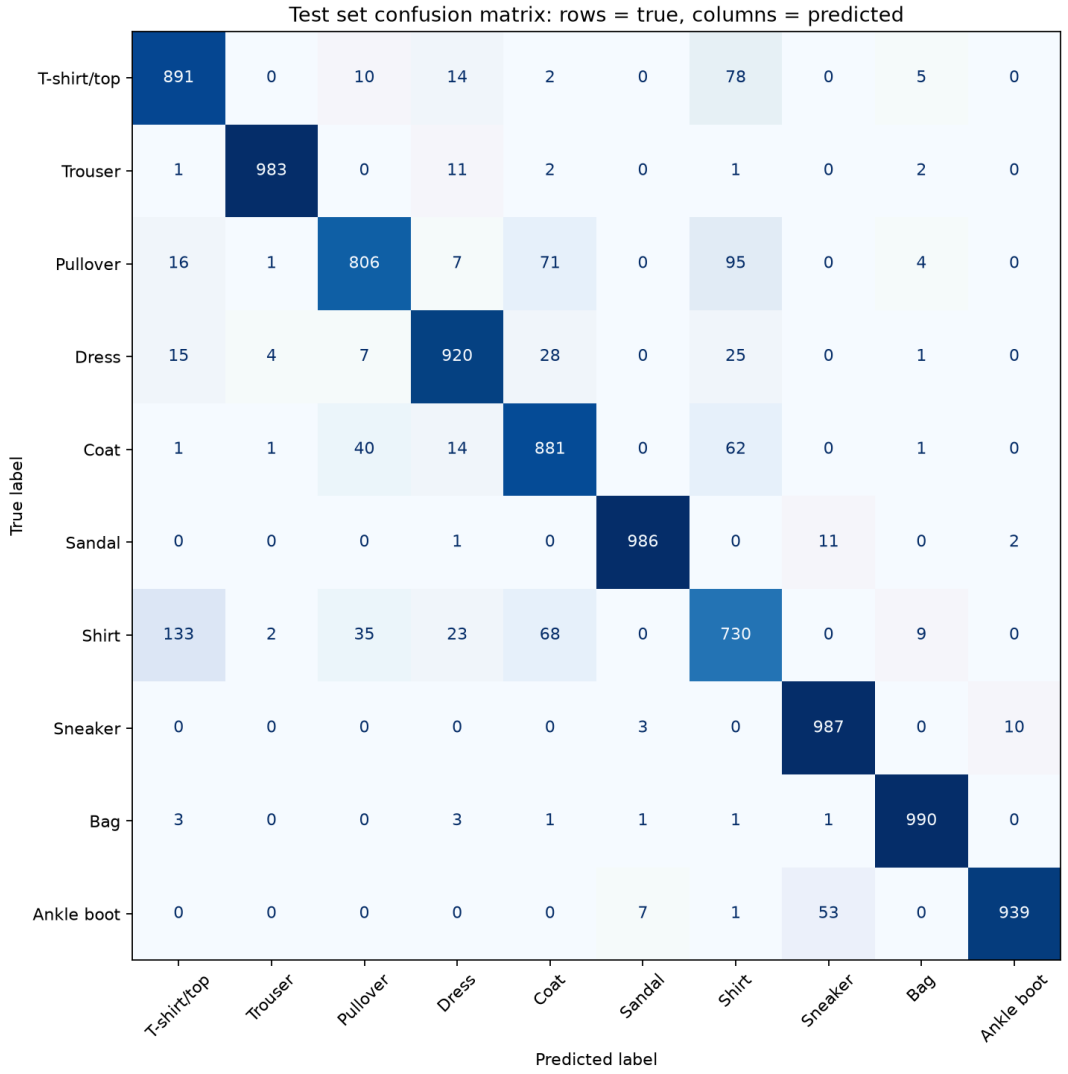


Figure 2. Test confusion matrix for the weights selected using the lowest validation loss. Rows are true classes and columns are predictions.

The diagonal contains correct predictions. Adding its values gives 9,113 correct predictions out of 10,000 test images, or 91.13 percent accuracy. This matrix describes the selected model, not necessarily the model at epoch 40.

Bags, sneakers, sandals and trousers are classified well: 990, 987, 986 and 983 correct predictions out of 1,000 for each class. Shirts are harder, with 730 correct predictions. The model predicts 133 shirts as T-shirts or tops, and 95 pullovers as shirts. Similar clothing shapes may help explain these errors.

Exercise 2

Part i Residual connections and gradients

A residual connection sends the input around the convolutional path and adds it to the output. A bottleneck block calculates output = F(input) + shortcut(input). F is the sequence of convolutional layers. The shortcut gives gradients a more direct path to earlier layers during backpropagation. This can make a deep network easier to train, but it does not guarantee better accuracy or prevent overfitting.

Part ii ResNet 50 model

As shown in the lesson, ResNet-50 uses bottleneck blocks with a 1 x 1 convolution, then a 3 x 3 convolution, then another 1 x 1 convolution. We use 3, 4, 6 and 3 blocks in the four stages. This gives 16 bottleneck blocks and 49 convolutional layers before the final classifier.

The first 1 x 1 convolution reduces the number of channels, so the 3 x 3 convolution needs less computation. The 3 x 3 convolution learns local clothing patterns, such as edges, sleeves, shoe shapes, or bag handles. The last 1 x 1 convolution expands the channels again. We use only ResNet-50 because ResNet-101 and ResNet-152 have many more blocks. They would take much longer to train, while the 28 x 28 Fashion-MNIST images and this exercise need one clear comparison with Exercise 1.

Part	Output size
Input	28 x 28 x 1
Small 3 x 3 stem	28 x 28 x 16
Stage 1: 3 blocks	28 x 28 x 64
Stage 2: 4 blocks	14 x 14 x 128
Stage 3: 6 blocks	7 x 7 x 256
Stage 4: 3 blocks	4 x 4 x 512
Global average pooling	512 values
Classifier	10 probabilities

The convolutions use zero padding at the image borders. When a shortcut needs more channels, we append channels filled with zeros. At the start of later stages, average pooling reduces the shortcut height and width before the zero channels are added. This lets the shortcut and the convolutional path have the same shape for addition.

Part iii Training results and comparison

Training time. The ResNet-50 run took 12,647.5 seconds. This is much more time than the Exercise 1 CNN run because ResNet-50 has 16 bottleneck blocks and 49 convolutional layers, while Exercise 1 has only three convolutional layers.

Overfitting. For this ResNet run, we cannot identify one clear epoch where overfitting starts. The validation and test loss and accuracy have many rises and falls instead of changing smoothly. These fluctuations make the curve harder to interpret than the Exercise 1 curve. We should therefore not make a strong conclusion about the exact beginning of overfitting from this single run.

Comparison with Exercise 1. In Exercise 1, the training loss continued to decrease after about epoch 6 while validation and test loss generally increased. That gave a clearer overfitting pattern. In the ResNet graph, training loss decreases, but validation and test results move up and down across many epochs.

Metric	Exercise 1 CNN	ResNet-50
Best validation mean accuracy	93.0%	93.6%
Best validation mean loss	0.21	0.25
Test mean accuracy	91.13%	89.79%
Test mean loss	0.25	0.29

From the result shown, the above table can identify that the 2 models achieve a very similar result. The model that used in exercise 1 achieved heights Test mean accuracy and lower mean and best loss but the ResNet achieved a little height best validation accuracy.

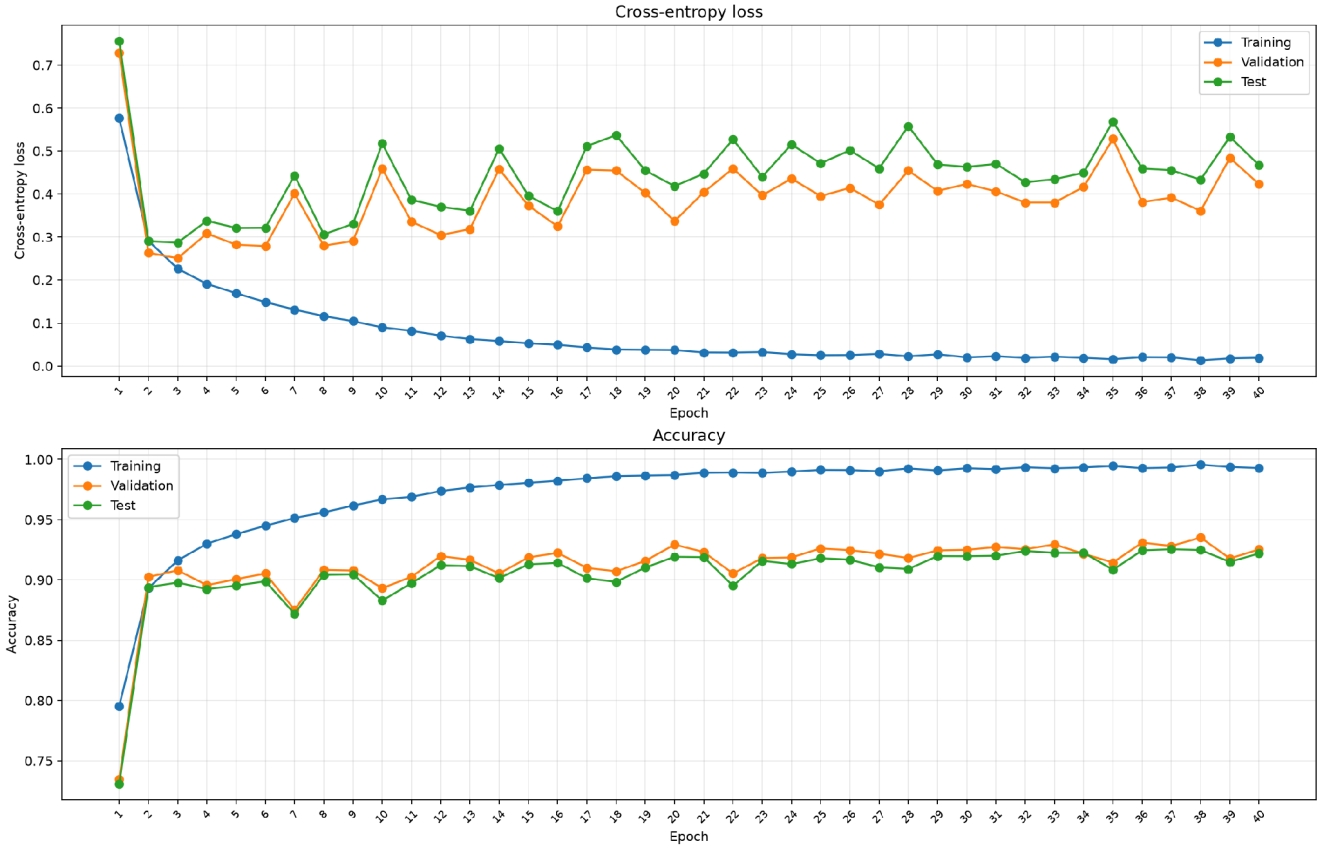


Figure 3. ResNet-50 loss and accuracy for training, validation and test data across all 40 epochs.

Why the ResNet curves are difficult to read

The training curves improve steadily, but the validation and test curves are not smooth. For example, validation and test loss rise and fall many times after the first few epochs, while validation and test accuracy stay around a similar level with repeated changes. This means that a single loss increase is not enough to identify a clear overfitting point. The selected weights still come from the epoch with the lowest validation loss.

ResNet test predictions by clothing class

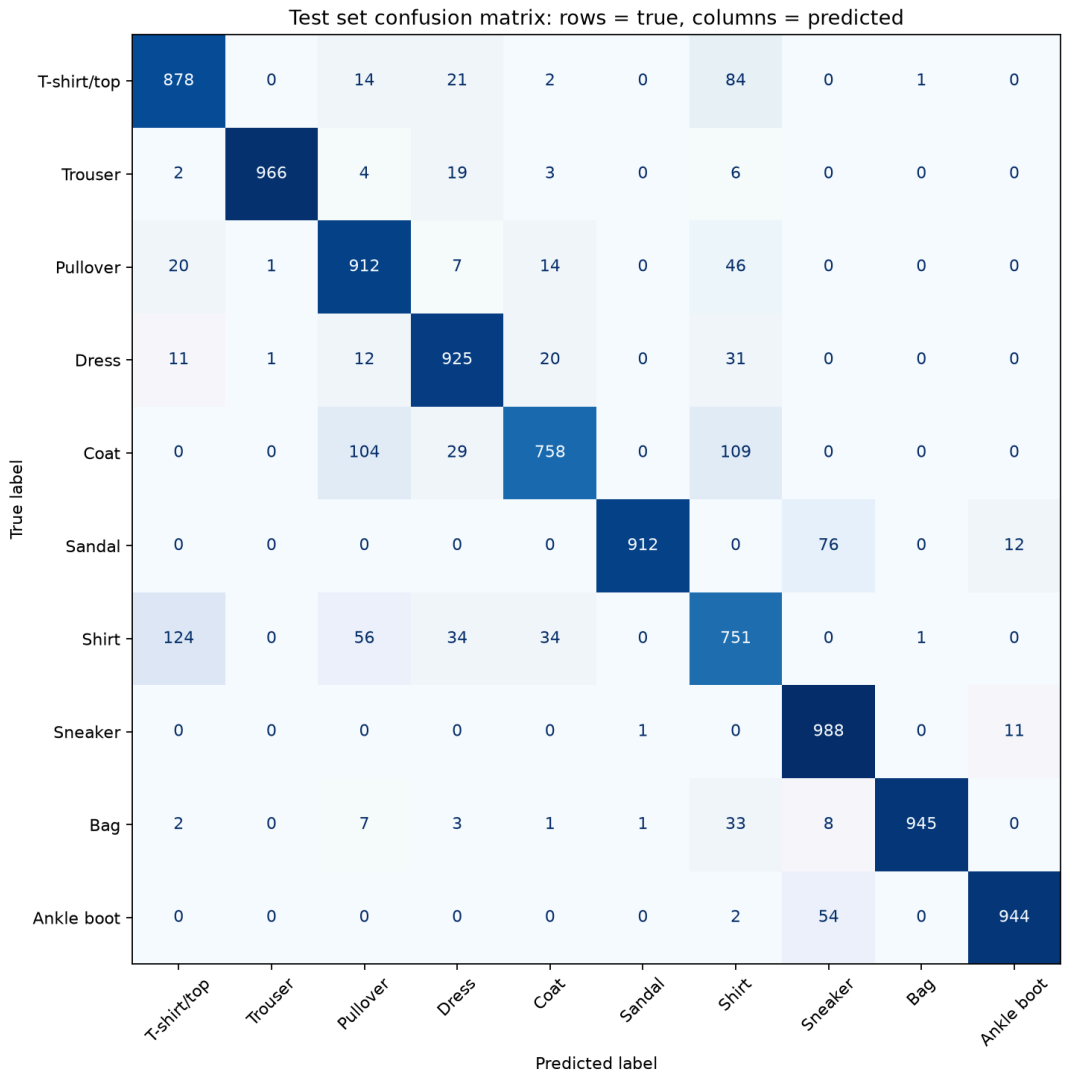


Figure 4. ResNet-50 test confusion matrix for the weights selected using the lowest validation loss. Rows are true classes and columns are predictions.

This confusion matrix shows which clothing classes the selected ResNet-50 model classifies correctly and which classes it confuses. It should be compared with the Exercise 1 confusion matrix together with the learning curves, rather than using training time alone to decide which model is better. As we can see both models confuse the shirts with the T-shirt/top, something that is expected because those two categories are very similar. But we can see that the ResNet also has trouble identifying.